

ARCHITECTURE DE RÉFÉRENCE

IMPLEMENTATION- READY SALES OS

Architecture globale scalable et élite — Prête pour le lancement

PROJET
SALES OS V1

VERSION
2.0 Consolidée

STATUT
Implementation-Ready

SECTION 01

Vision & Principes Fondateurs

SALES OS est une plateforme de gestion commerciale conçue pour orchestrer l'ensemble du cycle de vente — de la gestion des clients à l'enregistrement des commissions — dans un environnement multi-tenant sécurisé. L'architecture repose sur un principe fondamental : construire un **noyau commercial inébranlable** dont les frontières accueillent l'avenir.

Chaque module est classifié selon trois niveaux de maturité, ce qui permet de distinguer clairement ce qui doit fonctionner dès le premier jour, ce qui est structurellement prêt, et ce qui viendra dans les itérations suivantes.

Niveaux de Maturité

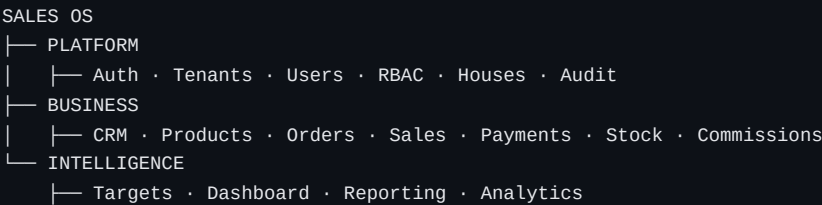
- IMPLÉMENTÉ** — Module entièrement fonctionnel en V1. Code, tests, API, UI — tout doit être opérationnel au lancement.
- PRÉPARÉ** — Modèle de données et/ou API prêts. Les champs existent en base, les contrats sont définis, mais la logique métier complète et l'interface ne sont pas encore développées.
- DIFFÉRÉ** — Non construit en V1. Les concepts sont documentés dans l'architecture pour guider les futures itérations, mais aucun code n'est écrit.

Le principe directeur est simple : **construire un noyau commercial rocher dont les frontières accueillent le futur**. Chaque module est isolé par des interfaces claires, chaque table porte le `tenant_id`, chaque événement est persisté avant d'être publié. L'architecture est conçue pour être étendue, jamais refactorisée en profondeur.

SECTION 02

Architecture Physique : Modular Monolith

SALES OS adopte une architecture de **monolithe modulaire** — trois domaines fonctionnels clairement séparés, mais déployés comme une seule application. Chaque module possède des limites strictes (interfaces typées, événements asynchrones), mais partage une base de données unique et un processus unique.



PLATFORM gère l'infrastructure transversale : authentification, multi-tenant, utilisateurs, RBAC, maisons (points de vente) et audit. C'est le socle sur lequel tout repose.

BUSINESS contient le cœur commercial : CRM (clients), produits, commandes, ventes, paiements, stock et commissions. C'est le noyau générateur de valeur.

INTELLIGENCE regroupe les capacités analytiques : objectifs, dashboard, reporting et analytics. Ce domaine consomme les données de BUSINESS sans impacter les transactions.

Pourquoi Monolithe Modulaire en V1 ?

Simplicité opérationnelle — Un seul déploiement, une seule base de données, un seul processus à monitorer.

Transaction unique — Les opérations critiques (commande → vente → paiement → stock) s'exécutent dans une seule transaction DB, garantissant la cohérence sans Saga ni compensation.

Itération rapide — Pas de overhead réseau inter-service, pas de sérialisation, pas de gestion de contrats API versionnés entre services déployés séparément.

Chemin d'extraction préservé — Les limites de modules sont des interfaces claires. Si un module doit être extrait en microservice, ses contrats sont déjà définis.

SECTION 03

Stack Technique

La stack est choisie pour maximiser la productivité tout en garantissant la sécurité multi-tenant au niveau base de données. Chaque composant a un rôle précis et irremplaçable.

COMPOSANT	TECHNOLOGIE	VERSION	RÔLE
Frontend	Next.js / React	15.x	Interface utilisateur et rendu SSR
Backend / Admin	Payload CMS	3.x	API auto-générée, administration, access control
Base de données	PostgreSQL	16.x	Source de vérité, RLS multi-tenant
ORM	Drizzle	0.x	Requêtes typées, migrations, utilisé par Payload
Cache / Events	Redis	7.x	Cache, jobs, transport d'événements
Stockage	S3-Compatible	—	Images, fichiers, exports
Runtime	Node.js	20.x LTS	Environnement d'exécution

SECTION 04

ADR-001 : Payload CMS + PostgreSQL RLS

Contexte : La sécurité multi-tenant exige une isolation des données au niveau base de données. Payload CMS fournit son propre système d'Access Control au niveau applicatif, mais cela ne suffit pas — un bug applicatif ou une requête Drizzle directe peut contourner l'isolation. PostgreSQL Row Level Security (RLS) est la dernière ligne de défense : même si l'application est compromise au niveau applicatif, la base refuse toute donnée cross-tenant.

Décision : Valider RLS via un PoC avant tout développement métier. Le PoC doit démontrer que l'isolation tenant fonctionne dans tous les scénarios d'accès, y compris les contournements potentiels.

Cas de test du PoC :

- SELECT cross-tenant — Un utilisateur du tenant B ne peut lire aucune donnée du tenant A
- INSERT cross-tenant — Impossible d'insérer une ligne avec un tenant_id différent du contexte
- UPDATE cross-tenant — Impossible de modifier une ligne d'un autre tenant
- DELETE cross-tenant — Impossible de supprimer une ligne d'un autre tenant
- Transaction avec SET LOCAL — Le contexte tenant est correctement propagé dans les transactions
- Connection pool reuse — Après libération d'une connexion, le contexte tenant est nettoyé
- Payload API bypass — Les requêtes via l'API Payload respectent l'isolation RLS
- Drizzle direct query — Les requêtes Drizzle directes sont filtrées par RLS
- Async job context propagation — Les jobs asynchrones propagent le contexte tenant correctement

Critère de succès : Un utilisateur du Tenant B ne récupère **JAMAIS** une donnée du Tenant A, même avec des paramètres de requête manipulés.

Statut : EN ATTENTE DE VALIDATION

Le PoC RLS est le prérequis absolument bloquant. Aucun développement métier ne doit commencer avant que ce PoC soit validé avec succès sur les 9 cas de test ci-dessus. L'ensemble de l'architecture de sécurité repose sur cette validation.

SECTION 05

ADR-002 : Monolithe Modulaire vs Microservices

Contexte : V1 a besoin de simplicité, d'itération rapide et d'un déploiement unique. Les microservices apportent de la complexité opérationnelle (orchestration, sérialisation, cohérence distribuée, observabilité multi-service) qui n'est pas justifiée pour un premier produit.

Décision : Adopter le monolithe modulaire. Les modules communiquent via des interfaces typées (synchrone) et des événements persistés (asynchrone), mais sont déployés ensemble.

Conséquences :

Avantages & Inconvénients

Avantages :

- Transaction DB unique — cohérence ACID native pour les opérations critiques
- Déploiement simple — un artefact, un processus, une base
- Limites de modules via interfaces — extraction future possible sans refactorisation
- Pas de overhead réseau inter-service — performances optimales
- Debugging simple — un seul call stack, un seul log stream

Inconvénients :

- Coupling potentiel — discipline requise pour respecter les limites de modules
- Scaling global — impossible de scaler un module indépendamment en V1
- Déploiement global — un changement nécessite un redéploiement complet

Mitigation : Le chemin d'extraction est préservé. Les interfaces de chaque module sont documentées. Si un module nécessite un scaling indépendant en V2+, il peut être extrait en microservice avec un effort mesuré.

SECTION 06

ADR-003 : Redis comme Transport d'Événements

Contexte : Les modules doivent communiquer de manière asynchrone pour le découplage — par exemple, lorsqu'une vente est complétée, le module Commissions doit calculer les commissions, le module Stock doit libérer les réservations, et le module Notifications doit informer les parties prenantes.

Décision : Utiliser Redis Pub/Sub comme transport d'événements. Redis n'est **JAMAIS** la source de vérité.

Principe fondamental : Persist First, Publish After

Les événements sont d'abord **persistés dans PostgreSQL** (table `order_events`, `audit_events`), puis **publiés vers Redis**. Les subscribers sont fire-and-forget.

Si Redis est indisponible, les événements sont toujours en base. Un processus de re-scan peut republier les événements non distribués.

Redis est un **transport**, pas un event store. La source de vérité est PostgreSQL, toujours.

SECTION 07

Défense en Profondeur

La sécurité de SALES OS repose sur **5 barrières concentriques**. Même si une barrière est compromise, les barrières suivantes continuent de protéger les données. C'est le modèle de défense en profondeur (defense in depth).

Authentication → RBAC → Tenant Context → Business Authorization → PostgreSQL RLS → Data

1. **Authentication** — Vérification de l'identité de l'utilisateur (JWT, session). Sans authentification valide, aucune requête n'atteint la logique métier.
2. **RBAC (Role-Based Access Control)** — Vérification des permissions de l'utilisateur selon son rôle (SuperAdmin, Admin, Manager, Agent, Caissier, Lecteur). Un Agent ne peut pas accéder aux fonctions d'administration.
3. **Tenant Context** — Le contexte tenant est injecté dans chaque requête via `set_tenant_context(tenant_id)`. Ce contexte est propagé à PostgreSQL via la session variable `app.current_tenant`.
4. **Business Authorization** — Règles métier au-delà du RBAC : un Manager ne voit que les données de sa maison, un Agent ne voit que ses propres commissions, un Caissier ne peut enregistrer que des paiements.
5. **PostgreSQL RLS** — Dernière ligne de défense. Même si toutes les barrières applicatives sont contournées, PostgreSQL refuse toute donnée d'un autre tenant. C'est la garantie absolue d'isolation.

SECTION 08

Matrice RBAC V1

La matrice RBAC définit les permissions de chaque rôle. ✓ = autorisé, ✗ = interdit. Les annotations entre parenthèses indiquent des restrictions supplémentaires au-delà du rôle.

ACTION	SUPERADMIN	ADMIN	MANAGER	AGENT	CAISSIER	LECTEUR
Gérer tenants	✓	✗	✗	✗	✗	✗
Gérer utilisateurs	✓	✓	✗	✗	✗	✗
Gérer maisons	✓	✓	✗	✗	✗	✗
Voir toutes maisons	✓	✓	✓	✗	✗	✗
Gérer agents	✓	✓	✓ (maison)	✗	✗	✗
Créer commandes	✓	✓	✓	✓	✗	✗
Voir commandes	✓	✓	✓	✓ (propres)	✓	✓
Valider commandes	✓	✓	✓	✗	✗	✗
Enregistrer ventes	✓	✓	✓	✓	✗	✗
Enregistrer paiements	✓	✓	✓	✗	✓	✗
Voir paiements	✓	✓	✓	✓	✓	✓

Gérer produits	✓	✓	✓ (maison)	✗	✗	✗
Gérer stocks	✓	✓	✓	✗	✗	✗
Voir commissions	✓	✓	✓	✓ (propres)	✗	✓
Valider commissions	✓	✓	✓	✗	✗	✗
Gérer objectifs	✓	✓	✓	✗	✗	✗
Voir dashboard	✓	✓	✓	✓	✓	✓
Voir analytics	✓	✓	✓	✗	✗	✓
Accès audit	✓	✓	✗	✗	✗	✗

SECTION 09

Modèle de Données : Core

Les tables Core constituent le socle multi-tenant et organisationnel. Chaque table porte tenant_id pour l'isolation RLS.

tenants

COLONNE	TYPE	CONTRAINTE
id	UUID	PK, DEFAULT gen_random_uuid()
name	VARCHAR(100)	NOT NULL
slug	VARCHAR(50)	NOT NULL, UNIQUE
status	ENUM	active, suspended, archived
settings	JSONB	DEFAULT '{}'
created_at	TIMESTAMPTZ	DEFAULT now()
updated_at	TIMESTAMPTZ	DEFAULT now()

users

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL

email	VARCHAR(255)	NOT NULL, UNIQUE per tenant
password_hash	VARCHAR(255)	NOT NULL
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
role	ENUM	super_admin, admin, manager, agent, cashier, viewer
status	ENUM	active, inactive, suspended
last_login	TIMESTAMPTZ	NULL
created_at	TIMESTAMPTZ	DEFAULT now()

houses

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
name	VARCHAR(100)	NOT NULL
code	VARCHAR(20)	NOT NULL, UNIQUE per tenant
address	TEXT	NULL
city	VARCHAR(100)	NULL
country	VARCHAR(2)	NULL
manager_id	UUID	FK → users, NULL
status	ENUM	active, inactive
created_at	TIMESTAMPTZ	DEFAULT now()

agents

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
user_id	UUID	FK → users, NOT NULL

house_id	UUID	FK → houses, NOT NULL
code	VARCHAR(20)	NOT NULL
commission_rate	DECIMAL(5,2)	DEFAULT 0
status	ENUM	active, inactive
created_at	TIMESTAMPTZ	DEFAULT now()

customers

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
house_id	UUID	FK → houses, NOT NULL
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	NULL
email	VARCHAR(255)	NULL
address	TEXT	NULL
client_account_id	UUID	NULL (PRÉPARÉ digital)
agent_referent_id	UUID	FK → agents, NULL (PRÉPARÉ digital)
order_source	ENUM	manual, digital, referral (PRÉPARÉ)
status	ENUM	active, inactive
created_at	TIMESTAMPTZ	DEFAULT now()

SECTION 10

Modèle de Données : Business

Les tables Business constituent le cœur transactionnel de la plateforme. Elles capturent l'intégralité du cycle commercial : produit → commande → vente → paiement → stock → commission.

products

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
house_id	UUID	FK → houses, NOT NULL
name	VARCHAR(200)	NOT NULL
sku	VARCHAR(50)	NOT NULL
description	TEXT	NULL
unit_price	DECIMAL(12,2)	NOT NULL
currency	VARCHAR(3)	DEFAULT 'USD'
category	VARCHAR(50)	NULL
status	ENUM	active, inactive, discontinued
created_at	TIMESTAMPTZ	DEFAULT now()

orders

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
house_id	UUID	FK → houses, NOT NULL
customer_id	UUID	FK → customers, NOT NULL
source	ENUM	client, agent, digital (PRÉPARÉ)
agent_id	UUID	FK → agents, NULL (apporteur)
seller_id	UUID	FK → users, NULL (vendeur)
status	ENUM	draft, formalized, confirmed, cancelled, completed
total_amount	DECIMAL(12,2)	DEFAULT 0
currency	VARCHAR(3)	DEFAULT 'USD'
notes	TEXT	NULL

created_at	TIMESTAMPTZ	DEFAULT now()
updated_at	TIMESTAMPTZ	DEFAULT now()

order_items

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
order_id	UUID	FK → orders, NOT NULL
product_id	UUID	FK → products, NOT NULL
quantity	INTEGER	NOT NULL, CHECK > 0
unit_price	DECIMAL(12,2)	NOT NULL
discount	DECIMAL(5,2)	DEFAULT 0
total	DECIMAL(12,2)	NOT NULL

order_events

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
order_id	UUID	FK → orders, NOT NULL
event_type	ENUM	created, formalized, confirmed, cancelled, converted, completed
payload	JSONB	DEFAULT '{}'
triggered_by	UUID	FK → users, NULL
created_at	TIMESTAMPTZ	DEFAULT now()

sales

COLONNE	TYPE	CONTRAINTE
id	UUID	PK

tenant_id	UUID	FK → tenants, NOT NULL
order_id	UUID	FK → orders, NULL
house_id	UUID	FK → houses, NOT NULL
agent_id	UUID	FK → agents, NULL
seller_id	UUID	FK → users, NULL
status	ENUM	pending, completed, refunded
total_amount	DECIMAL(12,2)	DEFAULT 0
created_at	TIMESTAMPTZ	DEFAULT now()

payments

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
sale_id	UUID	FK → sales, NOT NULL
amount	DECIMAL(12,2)	NOT NULL
method	ENUM	cash, mobile_money, bank_transfer, card
status	ENUM	pending, completed, failed, refunded
reference	VARCHAR(100)	NULL
paid_at	TIMESTAMPTZ	NULL
created_at	TIMESTAMPTZ	DEFAULT now()

stock

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
house_id	UUID	FK → houses, NOT NULL
product_id	UUID	FK → products, NOT NULL

quantity	INTEGER	DEFAULT 0
reserved	INTEGER	DEFAULT 0
available	INTEGER	GENERATED ALWAYS AS (quantity - reserved) STORED
updated_at	TIMESTAMPTZ	DEFAULT now()
UNIQUE(tenant_id, house_id, product_id)		

stock_movements

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
house_id	UUID	FK → houses, NOT NULL
product_id	UUID	FK → products, NOT NULL
type	ENUM	in, out, adjust, reserve, release
quantity	INTEGER	NOT NULL
reference	VARCHAR(100)	NULL
reason	TEXT	NULL
created_at	TIMESTAMPTZ	DEFAULT now()

commissions

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
sale_id	UUID	FK → sales, NOT NULL
agent_id	UUID	FK → agents, NOT NULL
type	ENUM	percentage, fixed
rate	DECIMAL(5,2)	NOT NULL
amount	DECIMAL(12,2)	NOT NULL

status	ENUM	calculated, validated, paid
calculated_at	TIMESTAMPTZ	NOT NULL
paid_at	TIMESTAMPTZ	NULL
created_at	TIMESTAMPTZ	DEFAULT now()

SECTION 11

Modèle de Données : Intelligence

Les tables Intelligence capturent les objectifs, l'audit et les métriques. Elles sont principalement en lecture pour le dashboard et les rapports, et alimentées par les événements du domaine Business.

targets

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
agent_id	UUID	FK → agents, NULL
house_id	UUID	FK → houses, NULL
period	VARCHAR(20)	NOT NULL (ex: "2026-Q1")
type	ENUM	revenue, orders, units
value	DECIMAL(12,2)	NOT NULL
achieved	DECIMAL(12,2)	DEFAULT 0
created_at	TIMESTAMPTZ	DEFAULT now()

audit_events

COLONNE	TYPE	CONTRAINTE
id	UUID	PK
tenant_id	UUID	FK → tenants, NOT NULL
user_id	UUID	FK → users, NULL
action	VARCHAR(50)	NOT NULL

entity_type	VARCHAR(50)	NOT NULL
entity_id	UUID	NOT NULL
changes	JSONB	DEFAULT '{}'
ip_address	INET	NULL
created_at	TIMESTAMPTZ	DEFAULT now()

Convention d'isolation RLS

Toutes les tables métier contiennent `tenant_id` pour l'isolation RLS. Les index composites (`tenant_id, id`) sont créés systématiquement pour garantir les performances de filtrage tenant dans chaque requête.

SECTION 12

Politiques RLS

Les politiques RLS sont la dernière ligne de défense de l'isolation multi-tenant. Elles sont activées sur chaque table métier et utilisent une variable de session PostgreSQL pour filtrer automatiquement toutes les requêtes.

```
-- Activation RLS par table
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE customers ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
-- ... (toutes les tables métier)

-- Politique d'isolation tenant
CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- Même politique pour chaque table métier
CREATE POLICY tenant_isolation ON customers
  USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- Fonction de set du contexte tenant
CREATE OR REPLACE FUNCTION set_tenant_context(p_tenant_id UUID)
RETURNS VOID AS
    BEGIN
    PERFORM set_config('app.current_tenant', p_tenant_id :: text, true);
    END;

LANGUAGE plpgsql SECURITY DEFINER;
```

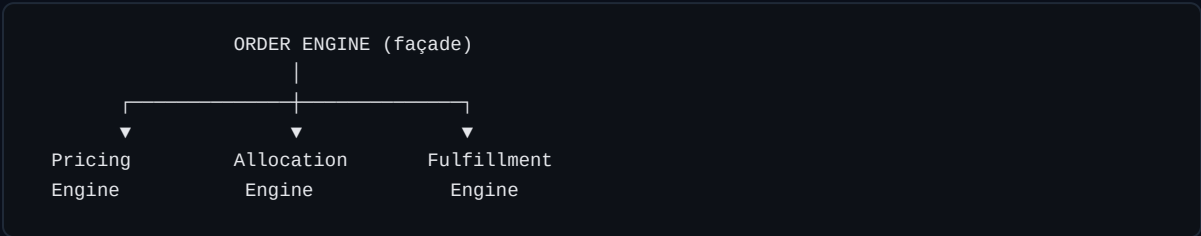
Obligation applicative

Avant chaque requête métier, l'application **DOIT** appeler `set_tenant_context(tenant_id)` pour positionner la variable de session PostgreSQL. C'est le fondement de l'isolation RLS — sans cette étape, les politiques RLS bloquent tout accès (denial-by-default).

SECTION 13

Order Engine : Façade Métier

L'Order Engine est la façade qui orchestre les trois sous-moteurs du cycle de commande. Le pattern façade garantit que la complexité interne est encapsulée — les modules appelants n'interagissent qu'avec une interface unifiée.



Pricing Engine V1 — Calcule les totaux à partir des `order_items`, applique les remises simples (pourcentage ou montant fixe par ligne). Le calcul est déterministe et reproductible.

Allocation Engine V1 — La maison (point de vente) est choisie manuellement par l'utilisateur lors de la création de la commande. L'allocation automatique (plus proche, meilleur stock) est différée en V2.

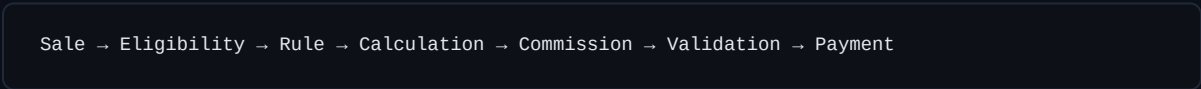
Fulfillment Engine V1 — Gère les transitions de statut : `draft` → `formalized` → `confirmed` → `completed`. Chaque transition est validée (règles métier), enregistrée dans `order_events`, et publie un événement asynchrone pour les side effects.

Les interfaces sont séparées pour permettre une évolution future sans refactoring — par exemple, le Pricing Engine V2 pourra intégrer des tarifs dynamiques sans impacter l'Allocation ou le Fulfillment.

SECTION 14

Commission Engine : Bounded Context

Le Commission Engine est un **bounded context isolé** — il possède son propre modèle de données, ses propres règles et ses propres invariants. Il communique avec le reste de l'application uniquement via les événements (`SALE_COMPLETED`) et l'interface typée du module Sales.



V1 — Commission simple : Pourcentage fixe par agent (`commission_rate` sur la table `agents`). Calcul automatique lors de la complétion d'une vente. Validation manuelle par un Manager ou Admin avant paiement.

V2 — Commissions avancées (différé) : Paliers progressifs, bonus sur objectifs, split commission entre apporteur et vendeur, règles par maison et par catégorie de produit.

SECTION 15

Triple Attribution des Transactions

Chaque transaction commerciale enregistre **trois dimensions d'attribution** qui permettent de mesurer indépendamment l'acquisition, la conversion et l'exécution de la vente.

- **Source** — L'origine de la transaction : client direct, agent apporteur, canal digital
- **Agent apporteur** — L'agent qui a apporté l'affaire (acquisition)
- **Vendeur** — L'utilisateur qui a exécuté la vente (conversion et closing)

Exemple de Triple Attribution

Commande #1001 — Source : CLIENT, Agent : Patrick, Vendeur : Marie, Maison : Kinshasa

Cette attribution permet de mesurer séparément : la performance d'acquisition (Patrick), la performance de conversion (Marie), le volume de ventes (Maison Kinshasa), et les commissions associées à chaque acteur.

Cette triple attribution est une caractéristique distinctive de SALES OS. Elle permet une analyse fine des performances qui n'est pas possible avec une attribution unique.

SECTION 16

Event Schema & Contrats

Chaque événement suit un schéma strict qui garantit la traçabilité, le debugging et la reproductibilité. Le schéma est versionné pour permettre l'évolution sans casser les subscribers existants.

```
{
  "event_id": "uuid",
  "event_type": "ORDER_CREATED",
  "aggregate_type": "order",
  "aggregate_id": "uuid",
  "tenant_id": "uuid",
  "timestamp": "ISO-8601",
  "version": 1,
  "payload": { ... },
  "metadata": {
    "triggered_by": "uuid",
    "correlation_id": "uuid",
    "source_module": "orders"
  }
}
```

Événements V1 :

- **ORDER_CREATED** — Commande créée (statut draft)
- **ORDER_FORMALIZED** — Commande formalisée (statut formalized)
- **ORDER_CONFIRMED** — Commande confirmée (statut confirmed)
- **ORDER_CANCELLED** — Commande annulée
- **ORDER_COMPLETED** — Commande complétée (statut completed)
- **SALE_CREATED** — Vente enregistrée
- **SALE_COMPLETED** — Vente complétée
- **PAYMENT_RECEIVED** — Paiement reçu
- **STOCK_MOVEMENT_CREATED** — Mouvement de stock enregistré

Persist First, Publish After

Les événements sont **persistés dans PostgreSQL** en premier (tables `order_events`, `audit_events`), puis **publiés vers Redis**. Redis est un **transport uniquement** — jamais la source de vérité. Si Redis est indisponible, les événements restent en base et peuvent être re-processés.

SECTION 17

Communication Inter-Modules

Deux modes de communication coexistent, chacun adapté à un cas d'usage précis :

Synchrone — Appel direct de fonction via interface typée. Utilisé pour les requêtes et les commandes au sein d'une même transaction. Garantit la cohérence immédiate.

Asynchrone — Événement via Redis Pub/Sub. Utilisé pour les side effects, les notifications et les réactions cross-module. Découplage temporel : l'émetteur ne dépend pas du consommateur.

DE	VERS	MODE	ÉVÉNEMENT/INTERFACE
Orders	Stock	Synchrone	reserveStock(orderId, items[])
Orders	Sales	Synchrone	createSaleFromOrder(order)
Sales	Payments	Synchrone	recordPayment(saleId, amount, method)
Sales	Commissions	Asynchrone	SALE_COMPLETED → calculate
Sales	Stock	Asynchrone	SALE_COMPLETED → commitReservation
Payments	Notifications	Asynchrone	PAYMENT_RECEIVED → notify
Any	Audit	Asynchrone	* → logAuditEvent

SECTION 18

Gestion d'Erreurs & Transactions

La stratégie de gestion d'erreurs distingue les opérations critiques (qui exigent la cohérence immédiate) des side effects (qui tolèrent l'asynchronisme et le retry).

Opérations business-critiques (commande → vente → paiement → stock → commission) utilisent des **transactions DB atomiques** au sein d'une seule boundary de module. Si une étape échoue, toute la transaction est rollback — pas de données partielles.

Side effects cross-module utilisent des **événements asynchrones avec retry** : 3 tentatives, backoff exponentiel (1s, 2s, 4s). Si un side effect échoue définitivement après 3 tentatives, il est enregistré dans `audit_events` avec `status=failed` pour résolution manuelle.

Boundaries de Transaction

Transaction 1 (synchrone, atomique) : Order → Order Items → Stock Reservation → Sale → Payment

Event 1 (asynchrone, retry) : SALE_COMPLETED → Commission Calculation

Event 2 (asynchrone, retry) : SALE_COMPLETED → Stock Commit

Event 3 (asynchrone, retry) : PAYMENT_RECEIVED → Notification

Pas de Saga en V1 — La complexité des compensations distribuées n'est pas justifiée. Les side effects sont conçus pour être idempotents et retryables.

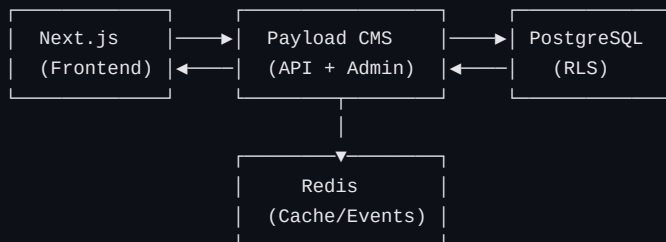
SECTION 19

Temps Réel & Déploiement

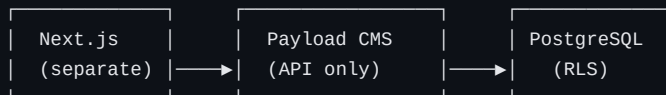
V1 — Polling Refresh : Le frontend utilise SWR (stale-while-revalidate) avec revalidation périodique. Simple, fiable, pas de connexion persistante. Suffisant pour les volumes V1.

V3 — WebSocket/SSE : Connexion persistante via Redis Pub/Sub → WebSocket/SSE. Push temps réel pour les dashboards, notifications et mises à jour de statut.

V1 Architecture:



V2+ Deployment Split:



En V1, Next.js et Payload sont déployés ensemble (single deployment). En V2+, le frontend Next.js peut être déployé séparément pour un scaling indépendant, le backend Payload restant sur sa propre infrastructure.

SECTION 20

Migrations, CI/CD & Observabilité V1

Migrations : Payload CMS gère son propre schéma via Drizzle. Les tables business personnalisées utilisent des fichiers de migration séparés dans `/src/migrations/`. L'ordre d'exécution est : migrations Payload en premier, puis migrations custom.

CI/CD : GitHub Actions → lint → test → build → deploy. Les tests incluent les tests d'isolation cross-tenant (RLS) qui échouent le build si une fuite de données est détectée.

Observabilité V1 :

- Logs structurés (JSON) — chaque log contient tenant_id, user_id, action, timestamp
- Health endpoint — /health vérifie DB, Redis, stockage S3
- Error tracking — Sentry ou équivalent pour la capture et l'agrégation des exceptions

Observabilité V2 (différé) : Métriques (Prometheus/Grafana), dashboards opérationnels, alertes proactives, distributed tracing.

SECTION 21

Production Readiness Checklist

Liste des prérequis avant la mise en production. Chaque item doit être validé et documenté.

- ☐ PoC Payload + PostgreSQL RLS validé
- ☐ Tests cross-tenant automatisés en CI/CD
- ☐ Modèle de données validé et migré
- ☐ Workflows V1 validés (Client → Commande → Vente → Paiement → Stock → Commission → Target)
- ☐ Matrice RBAC implémentée
- ☐ Backup et restauration testés
- ☐ Logs structurés opérationnels
- ☐ CI/CD pipeline fonctionnel
- ☐ Monitoring de base actif

SECTION 22

Scope V1 : Délimité & Réaliste

Le scope V1 est délibérément limité pour garantir la livraison dans les temps. Chaque module est classifié selon son niveau de maturité.

IMPLÉMENTÉ — V1

Auth · Tenants · Users · RBAC · Houses · CRM (Customers) · Products · Orders (+ Order Engine) · Sales · Payments · Stock · Commissions (+ Commission Engine simple) · Targets · Dashboard · Audit

PRÉPARÉ — Modèle + Champs

Digital Client (client_account_id, order_source, agent_referent_id) · Deployment Split · Event Schema

DIFFÉRÉ — V2+

Digital Commerce (panier, checkout) · Real-time push · Advanced Commissions (tiers, bonus) · Analytics avancés · Forecasting · IA/Recommandations · SaaS multi-entreprise commercialisé · Mobile · API publique

SECTION 23

Phase 0 : Pré-Ingénierie

Avant de coder, une phase de pré-ingénierie de 1-2 semaines est requise. Elle garantit que les fondations sont validées et que les décisions d'architecture sont confirmées avant l'investissement de développement.

- **PoC Payload + PostgreSQL RLS** — 3 jours
- **Modèle de données détaillé + validation** — 3 jours
- **Matrice RBAC + tests cross-tenant** — 2 jours
- **Spécifications fonctionnelles V1 workflows** — 4 jours
- **ADRs + contrats inter-modules** — 2 jours

Total : ~14 jours ouvrés

Investissement stratégique

La Phase 0 est un investissement de 2 semaines qui garantit que les 8-12 semaines de développement qui suivent ne seront pas remises en cause par un problème d'architecture fondamental. C'est l'assurance qualité la plus rentable du projet.

SECTION 24

Phase 1 : Implémentation V1

Planning réaliste basé sur l'expérience. Chaque semaine cible un périmètre défini avec des livrables mesurables.

SEMAINE	PÉRIMÈTRE	MODULES
1-2	Fondations	Auth, Tenants, Users, RBAC, RLS policies
3-4	Core CRM	Houses, Agents, Customers, Products
5-6	Moteur Commandes	Orders, Order Engine (Pricing, Allocation)
7-8	Ventes & Paiements	Sales, Payments, Fulfillment Engine
9-10	Stock & Commissions	Stock, Stock Movements, Commission Engine
11-12	Intelligence & Finalisation	Targets, Dashboard, Audit, Tests E2E, Déploiement

Total : 12 semaines (3 mois) pour un développeur senior, ou **6-8 semaines** pour une équipe de 2-3 développeurs.

Roadmap V1 → V4

La roadmap est progressive et réaliste. Chaque version s'appuie sur la précédente sans la remettre en cause.

V1 — Commercial Core (3 mois)

Multi-tenant, 14 modules, moteurs simples (Pricing, Allocation, Fulfillment, Commission), audit, dashboard de base. Le noyau commercial est opérationnel de bout en bout.

V2 — Operational Excellence (2-3 mois)

Commissions avancées (paliers, bonus, split), stocks avancés (alertes, réapprovisionnement), notifications temps réel, analytics détaillés, observabilité complète (métriques, dashboards, alertes).

V3 — Real-Time & Intelligence (2-3 mois)

Temps réel (WebSocket/SSE), event processing avancé, automatisations (règles métier, workflows), recommandations basées sur l'historique, forecasting prédictif.

V4 — Vision

Digital Commerce (panier, checkout, paiement en ligne), SaaS multi-entreprise commercialisé, IA avancée (recommandations, pricing dynamique), Mobile (app native), API publique pour intégrations tierces.

Score & Validation

Évaluation de l'architecture selon 8 dimensions critiques. Chaque score est justifié par rapport aux exigences du projet.

DIMENSION	CIBLE	JUSTIFICATION
Vision	9.5/10	Noyau commercial solide, frontières ouvertes
Modèle métier	9/10	Triple attribution, moteurs isolés
Architecture	9.5/10	Modular monolith, RLS, defense in depth
Sécurité	9/10	5 barrières, PoC RLS obligatoire
Multi-tenant	9.5/10	RLS + RBAC + tenant context

Scalabilité	9/10	Deployment split ready, extraction path
Opérations	8.5/10	Backup, CI/CD, logs — monitoring V2
Roadmap	9/10	V1 → V4 progressif et réaliste

Production-readiness : 7.5/10 → 9.5/10 après Phase 0 + validation PoC. L'écart est comblé par la validation des prérequis (PoC RLS, tests cross-tenant, backup testé) qui sont les conditions sine qua non du lancement.

VALIDATION

Validation du propriétaire

Approbation de l'architecture de référence

J'ai pris connaissance de l'architecture de référence ci-dessus et j'approuve le lancement de la Phase 0 (pré-ingénierie) et de la Phase 1 (implémentation V1) pour le projet SALES OS.

OBSERVATIONS

NOM DU PROPRIÉTAIRE

DATE

SIGNATURE

Architecture Implementation-Ready — SALES OS V1

Noyau commercial solide, frontières ouvertes, prêt pour l'ingénierie